**app/Http/Kernel.php** file's list of routes or global middleware.

Laravel will resolve a new instance of your middleware from the service container when you use the middleware's **terminate** function. utilize the singleton method of the container to register the middleware if you want to utilize the same instance of the middleware when the **handle** and **terminate** methods are invoked. Normally, this should be carried out in your **AppServiceProvider**'s register method:

```
use App\Http\Middleware\TerminatingMiddleware;
/**
 * Register any application services of the application Logic
 */
public function register(): void
{
 $this->app->singleton(TerminatingMiddleware::class);
}
```

# Conclusion

This chapter digs into Laravel's robust authentication system, concentrating on guards, providers, and several authentication techniques. Laravel's authentication system is built on the basic notion of guards and providers.

Authentication and authorization are critical when developing any software application. In the scenario of developing a Laravel application, security, user session management, authorized access, and password management become the critical methods to achieve security in our Laravel application.

Throughout the chapter, we looked into how the same can be achieved with hands-on examples to follow along and it is imperative to have these features in your Laravel application, especially when you will build a multi-user application in Laravel.

In the next chapter, we will look into API development in Laravel, and look into the RSET APIs, the Laravel Passport API, and much more.

# Points to Remember

- This chapter digs into Laravel's robust authentication system, concentrating on guards, providers, and several authentication